

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS**FACULTAD DE SISTEMAS E INFORMÁTICA****REQUISITOS DE DESARROLLO****ASIGNATURA**

Ingeniería de Requisitos

DOCENTE

Rodríguez Rodríguez, Ciro

INTEGRANTES GRUPO 5:

Alva Chacon, Jose Benjamin	24200045
Cordova Guerra, Josue Rodrigo	24200155
Sandoval Dominguez, Erick Marco	24200172
Melendez Bustamante, Alvaro Mathias	24200166

1. Introducción.....	3
Durante la fase de identificación y análisis de requisitos de desarrollo del proyecto OLYMPUS CORE, el equipo evaluó las restricciones técnicas, metodológicas y operativas necesarias para garantizar la construcción de una plataforma robusta, segura, escalable y alineada a las necesidades del gimnasio.....	3
El análisis permitió identificar factores críticos relacionados con concurrencia, validación de acceso mediante QR, rendimiento del sistema, seguridad de la información y compatibilidad tecnológica. Asimismo, se definieron estrategias de mitigación de riesgos y mecanismos de validación técnica orientados a garantizar la calidad del producto final.....	3
El desarrollo del proyecto se basa en metodologías ágiles y herramientas modernas de ingeniería de software, priorizando la construcción incremental del MVP, la trazabilidad de cambios y la validación continua de requisitos.....	3
2. Identificación y Análisis de Requisitos de Desarrollo:.....	3
1. Mapeo de Restricciones del Proyecto.....	3
1.2. Análisis de Riesgos.....	5
3. Especificación de requisitos de desarrollo.....	6
3.1. Requisitos de Desarrollo.....	6
3.2. Requisito Técnico TEC-001.....	7
3.3. Requisito Técnico TEC-002.....	9
4. Verificación de requisitos de desarrollo.....	11
4.1. Estrategias generales de validación.....	11
4.2. Inspección técnica.....	12
Requisito Original:.....	12
"El sistema backend debe registrar las reservas de las clases de spinning instantáneamente y bloquear la base de datos de forma segura para que no haya sobrecupos, soportando a todos los usuarios conectados a las 6:00 p.m.".....	12
5. Conclusiones.....	14

1. Introducción

Durante la fase de identificación y análisis de requisitos de desarrollo del proyecto OLYMPUS CORE, el equipo evaluó las restricciones técnicas, metodológicas y operativas necesarias para garantizar la construcción de una plataforma robusta, segura, escalable y alineada a las necesidades del gimnasio.

El análisis permitió identificar factores críticos relacionados con concurrencia, validación de acceso mediante QR, rendimiento del sistema, seguridad de la información y compatibilidad tecnológica. Asimismo, se definieron estrategias de mitigación de riesgos y mecanismos de validación técnica orientados a garantizar la calidad del producto final.

El desarrollo del proyecto se basa en metodologías ágiles y herramientas modernas de ingeniería de software, priorizando la construcción incremental del MVP, la trazabilidad de cambios y la validación continua de requisitos.

2. Identificación y Análisis de Requisitos de Desarrollo:

1. Mapeo de Restricciones del Proyecto

Durante esta etapa se identificaron las restricciones técnicas, temporales, metodológicas y operativas que condicionan el desarrollo del sistema.

Tipo de restricción	Descripción de la restricción	Justificación o fuente
Tecnológica	El sistema requiere la utilización de hardware específico para el control de acceso, como lectores biométricos y de códigos QR.	Análisis de requisitos del gimnasio.
Tecnológica	El Sistema de Gestión de Base de Datos (SGBD) será estrictamente PostgreSQL para asegurar integridad relacional y manejo	Decisión técnica

	eficiente de concurrencia.	
Temporal	El tiempo máximo para el desarrollo y entrega total del proyecto está fijado en 5 meses	Tiempo límite para presentación del proyecto
Proceso	El desarrollo se gestionará mediante la metodología ágil Extreme Programming (XP), exigiendo prácticas como Pair Programming, TDD y un cliente in-situ (o representante constante).	Metodología de desarrollo seleccionada
Seguridad	El sistema deberá implementar autenticación segura mediante JWT y cifrado bcrypt.	Requisitos no funcionales
Rendimiento	La validación de acceso QR deberá completarse en menos de 2 segundos.	RNF del sistema
Concurrencia	El sistema deberá soportar múltiples reservas simultáneas sin sobreventa de cupos.	Shadowing y análisis operacional
Escalabilidad	El sistema deberá soportar crecimiento futuro de socios y reservas.	Stakeholders
Compatibilidad	El sistema deberá funcionar en computadoras y dispositivos móviles.	Requisitos del producto
Integración	El sistema deberá permitir futuras integraciones con APIs externas y sistemas de pago.	Dueño del gimnasio

Elemento	Clasificación	Justificación
Desarrollo iterativo en Sprints de 1 semana	Requisito de Proyecto / Proceso	Define la frecuencia de los hitos de validación y la metodología de trabajo del equipo. No es una característica visible para el usuario final del gimnasio.
Uso de framework multiplataforma (React Native/Flutter)	Requisito de Proyecto (Restricción técnica)	Condiciona la selección de tecnologías y la arquitectura para abaratar costos, pero no define <i>qué</i> hace el sistema.
Integración obligatoria con el torniquete físico actual	Requisito de Proyecto (Restricción técnica)	Limita las decisiones de arquitectura de hardware del equipo de desarrollo, obligándolos a adaptarse a equipos preexistentes.
Control de versiones	Requisito de	Es una herramienta de trabajo exclusiva

Elemento	Clasificación	Justificación
con Git y GitHub	Proyecto / Proceso	para los programadores, afecta la gestión del código fuente.
Validación de código QR en menos de 2 segundos	Requisito del Producto (RNF)	Es lo que el sistema debe hacer y cómo debe comportarse de cara al usuario (repcionista y socio).
Registro de Ficha de Salud bajo Ley de Protección de Datos (Ley 29733)	Requisito de Dominio / Restricción legal	Es una normativa del negocio y del estado que el producto final debe cumplir obligatoriamente para poder operar.

1.2. Análisis de Riesgos

Restricción	Riesgo Identificado	Probabilidad	Impacto	Mitigación
Límite de tiempo (5 meses)	No completar el alcance total de los módulos operativos en el tiempo establecido.	Alta	Alto	Utilizar el Planning Game de XP para priorizar estrictamente el MVP. Realizar entregas pequeñas y frecuentes (ciclos semanales) para asegurar valor temprano.
Metodología XP (TDD y Pair Programming)	Resistencia del equipo a programar en pares o curva de aprendizaje lenta al escribir pruebas antes que el código (TDD), retrasando las primeras iteraciones.	Media	Medio	Rotación constante de parejas para compartir conocimiento. Mentoría técnica y adopción gradual del TDD en los componentes más críticos primero.
Base de datos PostgreSQL	Inconsistencias en el esquema de base de datos debido a cambios rápidos y refactorización constante propia	Baja	Alto	Implementar un sistema de control de versiones para la base de datos (ej. migraciones con Sequelize o Prisma) que se integre al pipeline de Integración Continua

	de XP.			
Integración QR	Incompatibilidad con hardware del gimnasio	Media	Alto	
Seguridad	Robo de credenciales o accesos indebidos	Media	Crítico	JWT, bcrypt y validaciones OWASP
Escalabilidad	Lentitud ante incremento de usuarios	Media	Alto	Arquitectura modular y optimización SQL
APIs externas	Caída de servicios integrados	Baja	Medio	Retries y manejo controlado de excepciones

3. Especificación de requisitos de desarrollo

Después de la identificación y análisis de restricciones, el equipo documentó detalladamente los requisitos técnicos necesarios para construir una plataforma robusta, escalable y alineada a los objetivos del gimnasio.

3.1. Requisitos de Desarrollo

1. El frontend será desarrollado utilizando React.js y Tailwind CSS para garantizar compatibilidad multiplataforma y diseño responsive.
2. El backend será implementado con Node.js y Express utilizando arquitectura basada en APIs REST.
3. PostgreSQL será utilizado como sistema gestor de base de datos relacional para asegurar integridad transaccional y soporte concurrente.
4. El sistema implementará autenticación segura mediante JWT y cifrado bcrypt para proteger credenciales y sesiones.
5. El módulo de reservas deberá soportar múltiples solicitudes concurrentes sin generar sobreventa de cupos.
6. El sistema integrará validación de acceso mediante código QR conectado al sistema de control de acceso del gimnasio.
7. El equipo utilizará GitHub como herramienta de control de versiones y colaboración.
8. El desarrollo seguirá metodología XP utilizando Pair Programming, TDD e integración continua.

9. Se implementarán pruebas unitarias y pruebas de concurrencia para validar estabilidad y rendimiento.
10. El sistema deberá mantener tiempos de respuesta menores a 2 segundos en procesos críticos.

3.2. Requisito Técnico TEC-001

ID: TEC-001

Título: Implementar módulo de reservas de clase en tiempo real

Tipo: Tarea técnica (backend + concurrencia)

Se necesita la implementación de un módulo backend que permita a los socios reservar clases grupales en tiempo real, validando de manera automática la disponibilidad de cupos, membresía activa y restricciones por penalización. Para ello, se deberá soportar múltiples reservas simultáneas sin pérdida ni sobrescritura de información, manteniendo actualizado el aforo de cada clase.

Criterios de aceptación (funcionales)

- Dado un usuario con membresía activa y cupos disponibles, el sistema registra la reserva correctamente.
- Después de cada reserva el aforo disponible se actualizará automáticamente por el sistema.
- Si la clase no tiene cupos disponibles, el sistema devolverá un mensaje explicando que el aforo está lleno.
- El sistema bloqueará la reserva por si un usuario tiene membresía vencida o penalización activa.
- Se registran todas las reservas en la base de datos con fecha, hora y usuario asociado.

- El sistema deberá soportar reservas simultáneas evitando duplicidad o inconsistencias.

Requisitos no funcionales

- Rendimiento: El proceso de reserva debe completarse en menos de 2 segundos.
- Concurrencia: El sistema debe soportar al menos 100 usuarios simultáneos.
- Disponibilidad: El módulo deberá mantenerse operativo el 99.9% del tiempo.
- Integridad: No debe existir sobrescritura ni pérdida de reservas.
- Compatibilidad: El sistema deberá funcionar en computadoras y dispositivos móviles.

Dependencias

- PostgreSQL configurado con control transaccional
- Módulo de autenticación implementado
- APIs REST configuradas
- Módulo de penalizaciones operativo
- Horario y aforos previamente registrados

Definition of Ready (DoR)

Criterio	Estado requerido	Responsable
Modelo de datos validado	Tablas de clases y reservas implementadas	Backend Team
APIs definidas	Endpoints documentados	Backend Team

Reglas de negocio definidas	Penalizaciones y aforos documentados	QA Team
Diagramas UML disponibles	Casos de uso y secuencias completados	Análisis
Casos de prueba	Escenarios concurrentes definidos	Product Owner
Entorno backend operativo	APIs funcionando correctamente	Backend Team

Definition of Done (DoD)

Criterio	Verificación
Reservas operativas	Registro correcto de reservas
Validación de aforo	Actualización automática funcionando
Manejo concurrente	Sin duplicidad de reservas
Pruebas exitosas	Escenarios concurrentes probados
Logs implementados	Auditoría de reservas disponible
Seguridad validada	Autenticación funcionando
Documentación actualizada	APIs y endpoints documentados
Revisión técnica	Validación realizada por el equipo

3.3. Requisito Técnico TEC-002

ID: TEC-002

Título: Implementar módulo de validación de acceso mediante código QR

Tipo: Tarea técnica (backend + integración QR)

Implementación de servicio backend que reciba códigos QR generados desde la aplicación del socio, consulte el estado de la membresía en tiempo real y permita o bloquee automáticamente el acceso al gimnasio mediante integración con el sistema de torniquete.

El módulo deberá conectarse con PostgreSQL, validar la vigencia de la membresía y registrar cada intento de acceso en logs estructurados para fines de auditoría y trazabilidad.

Criterios de aceptación (funcionales)

- Dado un código QR válido y una membresía activa, el sistema permitirá el acceso y devolverá el estado que indica su aprobación.
- Dado un QR inválido o una membresía vencida, el sistema devolverá el estado de su membresía expirada.
- El sistema registra fecha, hora, usuario y resultado de validación en la base de datos.
- Luego de que se haya aprobado el acceso, el sistema envía una señal de apertura al torniquete.
- Dado un error de conexión o timeout (>5s), el sistema intentará hasta 2 veces antes de devolver un mensaje de “validation service unavailable”.
- El sistema registrará logs estructurados (INFO, WARN, ERROR) para cada intento de acceso.

Requisitos no funcionales

- Seguridad: Las validaciones deberán utilizar autenticación JWT y cifrado bcrypt.
- Rendimiento: El tiempo promedio de validación debe ser menor a 2 segundos.
- Disponibilidad: El módulo debe mantenerse operativo el 99.9% del tiempo.
- Trazabilidad: Todos los accesos deberán almacenarse en logs de auditoría.
- Escalabilidad: El sistema debe soportar más de 100 usuarios concurrentes.

Dependencias

- Base de datos PostgreSQL y operativa
- Integración con lector QR y torniquete
- APIs backend configuradas.
- Variables de entorno JWT definidas
- Conexión estable a red local del gimnasio

Definition of Ready (DoR)

Criterio	Estado requerido
Diagramas UML disponibles	Diagramas de secuencia y actividades completados
Base de datos modelada	Tablas y relaciones validadas
Endpoints definidos	APIs documentadas y revisadas
Validaciones definidas	Reglas de membresía documentadas
Entorno de desarrollo	Backend y frontend configurados
Casos de prueba	Escenarios de éxito y error definidos
Seguridad documentada	JWT y cifrado especificados

Definition of Done (DoD)

Criterio	Verificación
Código implementado	Funcionalidad integrada correctamente
Validación QR operativa	Acceso permitido y denegado funcionando
Pruebas unitarias	Todas las pruebas exitosas
Logs implementados	Auditoria visible en pruebas
Manejo de errores	Excepciones controladas
Seguridad validada	Contraseñas cifradas y JWT funcional
Documentación técnica	APIs y configuraciones actualizadas
Revisión de código	Validación realizada por el equipo

4. Verificación de requisitos de desarrollo

4.1. Estrategias generales de validación

- **Revisiones de Pares (Peer Reviews):** Se realizarán inspecciones técnicas estáticas mediante grupos de desarrolladores para validar que los requisitos documentados (DoR y DoD) sean comprendidos unánimemente.
- **Prototipos Arquitectónicos (Spikes):** Antes de programar el sistema completo, se desarrollará un prototipo rápido (piloto) del escaneo del código QR y su comunicación con el torniquete físico para validar la compatibilidad de hardware.
- **Pruebas de Estrés y Rendimiento:** Se ejecutarán simulaciones de carga concurrente en el entorno de desarrollo para asegurar que el módulo de reservas soporte la demanda sin bloquear la base de datos (verificando latencias menores a 2 segundos).

- **Auditorías de Seguridad:** Se ejecutarán pruebas de vulnerabilidades basadas en las guías de OWASP para validar que la "Ficha de Salud" y los datos de pago estén correctamente cifrados.
- **Verificación Automatizada (CI/CD):** Se auditará el cumplimiento de los estándares de codificación insertando herramientas de análisis estático (Linters) directamente en el pipeline de validación de GitHub.

4.2. Inspección técnica

Requisito Original:

"El sistema backend debe registrar las reservas de las clases de spinning instantáneamente y bloquear la base de datos de forma segura para que no haya sobrecupos, soportando a todos los usuarios conectados a las 6:00 p.m."

Atributo	Evaluación	Observaciones	Recomendación
Atomicidad	No atómico	Mezcla dos preocupaciones distintas: (a) el tiempo de respuesta del sistema ("instantáneamente") y (b) el control de concurrencia y sobrecupos en la base de datos. Deben separarse.	Separar ambos conceptos
Testabilidad	No testable	Palabras como "instantáneamente" y "todos los usuarios" no son cuantificables. El equipo de pruebas no puede medir "todos", necesita un número máximo de peticiones simultáneas (ej. 200 req/sec) para configurar la prueba de	Definir métricas cuantificables

		estrés.	
Factibilidad	Parcialmente factible	Node.js y PostgreSQL (las tecnologías elegidas en los RD) pueden manejar esto, pero "bloquear la base de datos" entera no es factible ni eficiente; arruinaría el rendimiento de todo el gimnasio. Debe especificarse que el bloqueo es solo para la <i>fila</i> de la clase específica.	Utilizar bloqueo por fila
No ambigüedad	Ambiguo	Términos como "de forma segura" son vagos. ¿Se refiere a seguridad contra hackers (OWASP) o seguridad transaccional (ACID)?	Especificar seguridad transaccional

Requisito Técnico Reescrito y Validado: Al detectar las deficiencias y la falta de parámetros matemáticos para las pruebas automatizadas, el requisito se reescribe de forma medible para el equipo de desarrollo:

- **ID:** TEC-03
- **Descripción:** El endpoint de creación de reservas (POST /reservas) en el backend debe implementar bloqueos a nivel de fila utilizando transacciones de PostgreSQL para prevenir condiciones de carrera (*Race Conditions*) al verificar y actualizar el aforo máximo de una clase.
- **Criterio de Aceptación (Meta matemática):** Una prueba de carga automatizada en el entorno de desarrollo simulará 150 peticiones concurrentes en el mismo segundo apuntando a una clase con un aforo máximo de 20 personas.

- **Condición de éxito:** El sistema debe registrar exactamente 20 reservas exitosas (HTTP 201) y 130 rechazos por clase llena (HTTP 409), manteniendo un tiempo de respuesta (Percentil 95) menor a 500 ms para el lote completo.

- **Validaciones Técnicas Esperadas**

Validación	Resultado esperado
Control concurrente	No existe sobreventa de cupos
Integridad de datos	No existen reservas duplicadas
Transacciones PostgreSQL	Operativas correctamente
Respuestas HTTP	Correctamente diferenciadas
Rendimiento	Percentil 95 $\leq$ 500 ms

- **Riesgo Mitigado**

Este requisito técnico permite mitigar:

- Pérdida de integridad del afor
- Sobreventa de clases
- Inconsistencias concurrentes
- Corrupción de datos
- Errores de sincronización entre usuarios simultáneos

5. Conclusiones

- Los requisitos de desarrollo permitieron identificar restricciones técnicas, metodológicas y operativas críticas para el proyecto OLYMPUS CORE.

- La aplicación de metodologías ágiles y validaciones técnicas mejora la calidad y trazabilidad del desarrollo.
- La verificación técnica permitió transformar requisitos ambiguos en requisitos medibles y verificables.
- La implementación de control concurrente, integración QR y seguridad basada en JWT fortalece la confiabilidad y escalabilidad del sistema.
- El enfoque incremental basado en MVP facilita la construcción progresiva del sistema y reduce riesgos durante el desarrollo.